GSOC 2026 PROPOSAL - FOSSASIA

Improve the Magic ePaper Badge app into a User friendly Functional App

---

ABSTRACT

The Magic ePaper Badge is an innovative hardware platform. It features a tri-color ePaper display, NFC communication, low power usage, and portable badge usability. This creates a unique interactive system for makers, developers, event communities, and hardware enthusiasts. Though the existing open-source Flutter companion app lays the groundwork for badge interaction, the user experience, workflow, editing tools, and NFC transfer reliability need significant improvements. This project aims to enhance the functionality and usability of the Magic ePaper Badge application, transforming it into a production-ready, contributor-friendly, and intuitive cross-platform solution. The main focus of this project is to redesign the application to support a modern user-friendly editing workflow. Users should easily create customized badge content using freehand drawing tools, text insertion, stickers, imported

images, and visual effects within a smooth and responsive Flutter interface. A significant technical challenge is ensuring that all created content is optimized for the tri-color ePaper display, as standard RGB rendering techniques are not suitable. To solve this, the project will introduce an intelligent image processing and palette conversion pipeline that can manage contrast balancing, brightness adjustments, threshold tuning, image rotation, cropping, scaling, and color reduction tailored for the badge hardware. Another important goal is to improve the NFC-based content transfer pipeline. This pipeline connects the mobile or desktop application to the physical badge. The existing workflow will be refined to include stable device detection, reliable packetized data transfer, safe writing mechanisms, validation after transfer, and visible progress updates. These enhancements will make the transfer process reliable for real-world use. From a software engineering viewpoint, the proposal also highlights the need for maintaining the codebase, modular design, and scalability for contributors. The Flutter application will be organized into clean layers for presentation, state management, rendering, and device communication, simplifying feature extensions for future contributors. Additionally, the project will implement thorough widget tests, integration tests for NFC workflows, automated release processes through CI/CD, improvements in platform packaging, contributor documentation, and architecture diagrams. By the end of the GSoC coding period, the final outcome will be a fully optimized, feature-rich, stable, and user-friendly Magic ePaper Badge application. It will significantly enhance both user creativity workflows and the long-term maintainability of the open-source project.

PROBLEM STATEMENT

The Magic ePaper Badge can be interacted with using the current application's basic features, but there are a few key areas that require improvement:

- UI requires more user-friendly workflows and improved usability.
- Richer editing support is required for text and drawing tools.
- NFC gearbox ought to be more dependable.
- Optimisation is required for tri-color ePaper image rendering.
- Cross-platform support requires stabilization
- Contributor onboarding and long-term maintainability are impacted by current bugs.
- These problems will be resolved by this project, which will also prepare the app for production.

PROPOSED FEATURES

1. Modern UI/UX Redesign

- Responsive layouts and a cleaner Flutter user interface
- Improved drawing, text, emoji, and image editing navigation
- Fast switching between colour palettes
- Redo/undo history stack
- Enhanced workflow for project save/load

2) Sophisticated Tools for Drawing and Editing

- Support for freehand drawings
- Control over brush size and opacity
- Shape tools (line, circle, and rectangle)

- Text insertion with alignment, style, and font size
- Emoji and sticker placement
- Layer-like object editing flow

3) Intelligent Image Enhancement for ePaper

- Bring in PNG, JPG, and WebP pictures
- Automatic tri-color palette conversion
- Adjusting contrast, brightness, and threshold
- Aspect lock, rotation, crop, and resize
- Prior to badge transfer, preview

4) Dependable Transfer of NFC Content

- Quicker device identification
- Data packet transfer that is stable
- Retry method for unsuccessful writes
- Progress indicator while synchronising
- After writing, transfer validation

5) Deployment Across Multiple Platforms

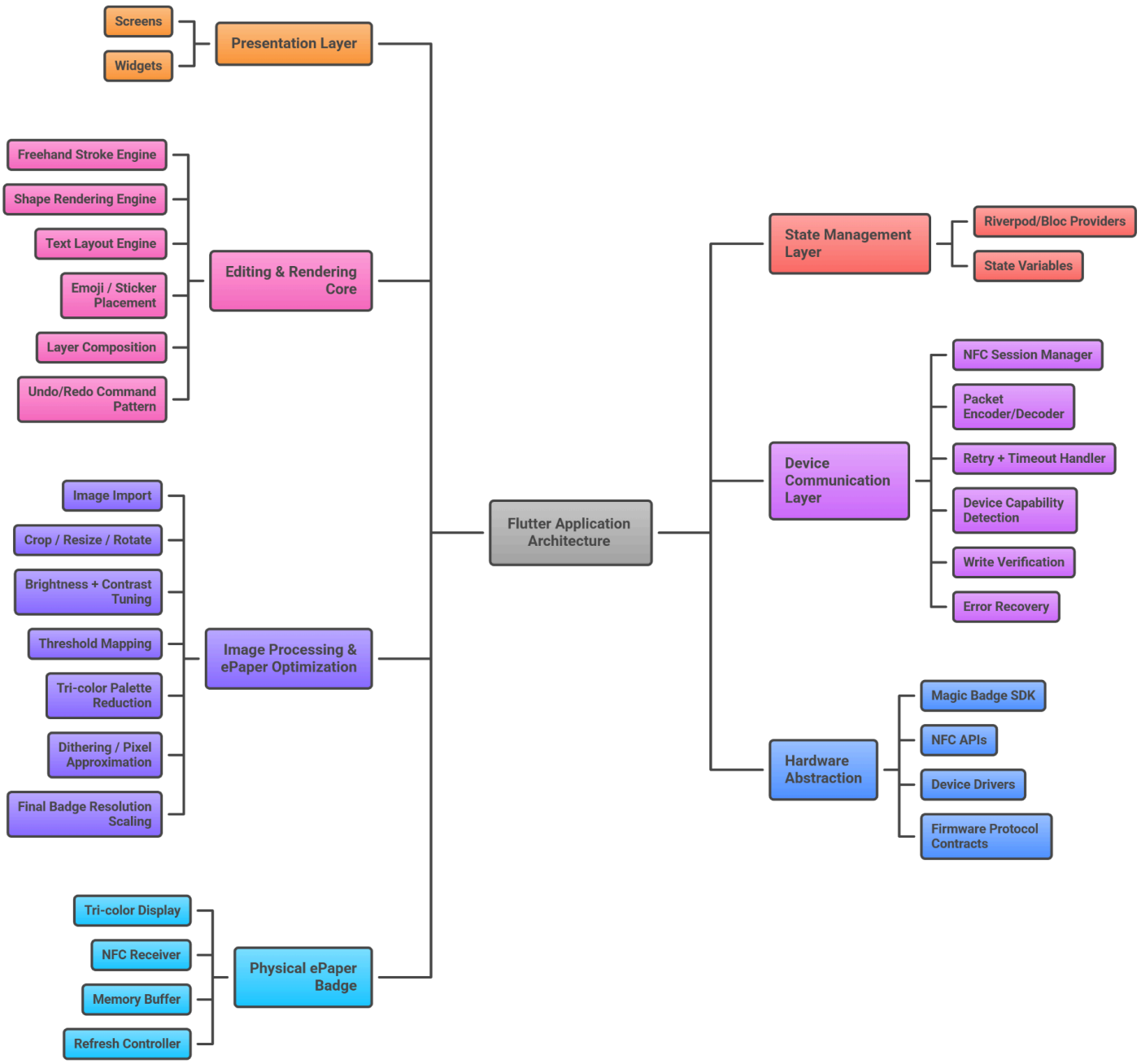
- Optimisation for iOS and Android
- Linux, macOS, and Windows builds
- When feasible, web preview compatibility
- CI/CD for releases using GitHub Actions

6) Documentation + Testing

- UI tool widget tests
- NFC workflow integration tests
- Setup instructions for contributors
- Module documentation and architecture diagrams
- Public relations guidelines and templates

Technical Architecture

Flutter Application Architecture



Made with Napkin

Module-Level Design

- presentation/ → Flutter screens, widgets, routing, responsive layouts
- core/editor/ → canvas engine, brush logic, shapes, text,
- emoji placement core/rendering/ → tri-color conversion, palette logic,
- image scaling services/nfc/ → transfer manager, retry-safe writes,
- validation state/ → Riverpod/Bloc providers for editor state and transfer progress platform/ → Android/iOS/Desktop platform-specific integrations tests/ → widget tests,
- rendering golden tests, and NFC integration mocks

Technical Implementation (Code-Level Design)

This part talks about how the suggested changes can be made to the actual repository. The code examples below are meant to be examples of how the project will be built, showing the engineering direction of the project

while still allowing for mentor feedback during community bonding.

1. Editor State Controller

A centralised state controller will handle the editor workflow, making sure that drawing actions, text elements, stickers, and imported images stay in sync across the UI.

```
class EditorController extends StateNotifier<CanvasState> {
  EditorController() : super(CanvasState.initial());

  void addStroke(Stroke stroke) {
    state = state.copyWith(
      strokes: [...state.strokes, stroke],
    );
  }

  void addTextElement(String value, Offset position) {
    final element = TextElement(
      text: value,
      position: position,
    );

    state = state.copyWith(
      textElements: [...state.textElements, element],
    );
  }

  void clearCanvas() {
    state = CanvasState.initial();
  }
}
```

Why this implementation

- Keeps the logic of the UI separate from the rendering
- Makes it easier to test
- Works well with undo and redo support
- Easy to use with Riverpod/Bloc

2. Undo and Redo Workflow

There will be a command-based action history so that you can safely undo any operation on the canvas.

```
abstract class CanvasCommand {
  void execute(CanvasState state);
  void revert(CanvasState state);
}

class AddTextCommand extends CanvasCommand {
  final TextElement element;

  AddTextCommand(this.element);

  @override
  void execute(CanvasState state) {
    state.textElements.add(element);
  }

  @override
  void revert(CanvasState state) {
    state.textElements.removeLast();
  }
```

Why this implementation

- Allows for a scalable editing history
- Safe for the future for layers and shape tools
- Makes the user experience a lot better
- Follows the design patterns of the production editor

3. Tri-Color Rendering Engine

The rendering layer creates a precise tri-color buffer that is compatible with badges from imported or drawn.

```
class EpaperRenderer {
  Uint8List render(ui.Image image) {
    final pixels = extractPixels(image);

    final converted = pixels.map((pixel) {
      if (pixel.red > 200) return BadgeColor.red;
      if (pixel.computeLuminance() > 0.5) {
        return BadgeColor.white;
      }
      return BadgeColor.black;
    }).toList();

    return packDisplayBuffer(converted);
  }
}
```

Why this implementation

- optimised for the limitations of ePaper displays
- maintains legibility on three-color screens
- reduces the size of the transfer payload.
- ensures accurate preview rendering
-

4. Reliable NFC Transfer Layer

A retry-safe service layer will be used in the NFC workflow to increase the dependability of hardware communication.

```dart
class BadgeTransferService {
  Future<bool> transfer(Uint8List payload) async {
    const maxAttempts = 3;

    for (int i = 0; i < maxAttempts; i++) {
      final success = await _send(payload);
      if (success) return true;
    }

    return false;
  }

  Future<bool> _send(Uint8List payload) async {
    final session = await NfcManager.instance.startSession();
    await session.write(payload);
    return await session.verify();
  }
}
```

Why this implementation

- Write flow that is safe to try
- Improved compatibility with hardware
- Production-level error management
- Increased user confidence while syncing

5. Image Optimization Pipeline

To enhance the badge's output quality, imported images will be normalised before rendering.

```
class ImageOptimizer {
  List<BadgeColor> optimize(List<Pixel> pixels) {
    return pixels.map((pixel) {
      final brightness =
          (pixel.red + pixel.green + pixel.blue) / 3;

      if (brightness > 180) return BadgeColor.white;
      if (pixel.red > 200) return BadgeColor.red;
      return BadgeColor.black;
    }).toList();
  }
}
```

Why this implementation

- Improved contrast on electronic papers
- Improved rendering of imported photos
- increases the visibility of text and logos
- Improved output of real-world badges

IMPLEMENTATION PLAN

Getting to Know Each Other (Before Coding)

- Look into the architecture of a study repository.
- Set up the local environment
- Talk to your mentors about the roadmap.
- Fix one or two problems with the starter
- Set the final limits for the module

Phase 1 (Weeks 1–3)

- The basis for the UI redesign
- Change the screens and navigation
- Add a drawing toolbar
- Make state management better.

Phase2 (Weeks 4–6)

- Text editor with emoji support
- Tools for importing and changing images
- Engine for converting three colours
- Rendering a preview in real time
  -

Phase 3 (Weeks 7–9)

- Refactor NFC transfer
- Retry and validation flow
- More devices that work with it
- Better ways to report errors

Phase 4 (Weeks 10–12)

- Packaging that works on all platforms
- Automated CI builds

- Testing widgets and integrations
- Guides for contributors and documentation

Things to deliver

- Flutter app ready for production
- Drawing and text tools that are more advanced
- Transfer of NFC badges that works
- Engine for rendering three colours at once
- Pipeline for releasing on multiple platforms
- Complete documentation for contributors

TECHNICAL SKILLS

I am good at:

- C and C++
- Java
- HTML, CSS, and JavaScript
- Understanding Flutter UI
- Design for UI and UX
- Interest in contributing to open source

I really want to help the Magic ePaper Badge ecosystem in the long term and make it easier to use and better for contributors

---

Expected Effect

This project will make the Magic ePaper Badge platform much easier to use by making it easy for end users to create content that is visually appealing and trustworthy. It will also make it easier for people who want to add to the repository in the future.
The final result will make it easier for both hardware fans and app developers to use the badge.

Contact Information

Name: TANGUTURI JAYASANKAR
Email: jayasankartanguturi@gmail.com